

stat462

2026-02-18

```
library(ggplot2)
library(tidyr)
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.2.0      v readr      2.1.6
## v forcats    1.0.1      v stringr   1.6.0
## v lubridate  1.9.5      v tibble    3.3.1
## v purrr      1.2.1
## -- Conflicts ----- tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

Summary Dataset:

```
diabetes <- read.csv(file="/Desktop/diabetes2.csv",header=T)

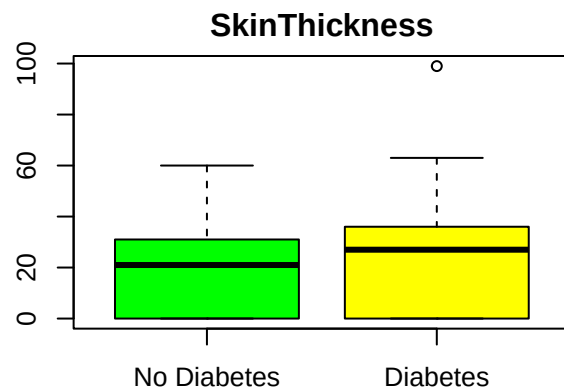
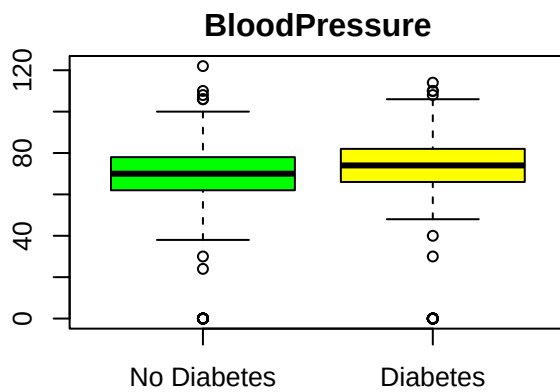
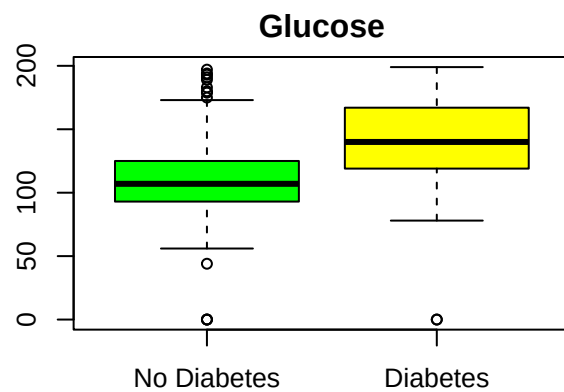
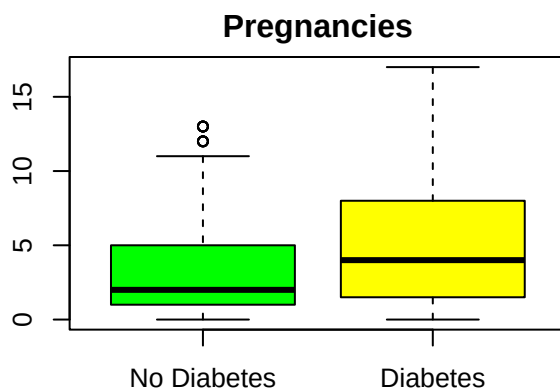
summary(diabetes)
```

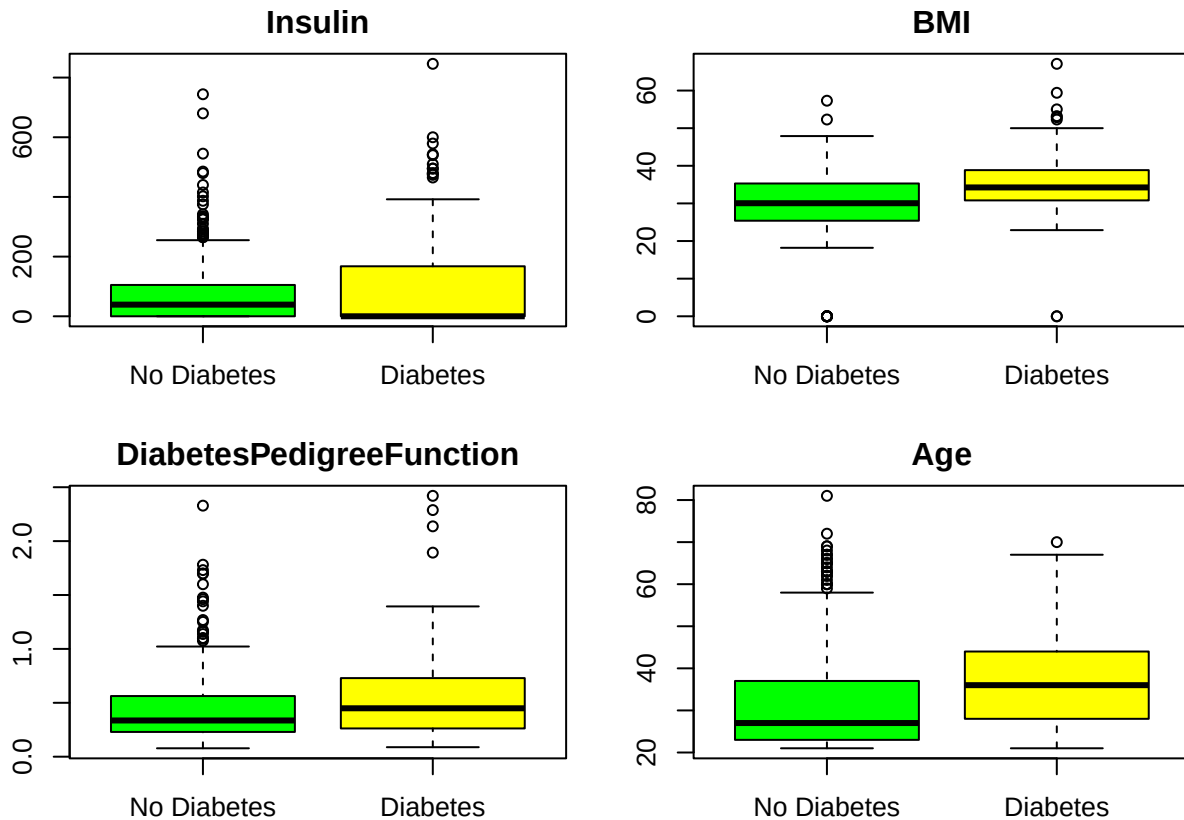
```
##   Pregnancies      Glucose      BloodPressure      SkinThickness
##   Min.   : 0.000   Min.   : 0.0   Min.   : 0.00   Min.   : 0.00
##   1st Qu.: 1.000   1st Qu.: 99.0   1st Qu.: 62.00   1st Qu.: 0.00
##   Median : 3.000   Median :117.0   Median : 72.00   Median :23.00
##   Mean   : 3.845   Mean   :120.9   Mean   : 69.11   Mean   :20.54
##   3rd Qu.: 6.000   3rd Qu.:140.2   3rd Qu.: 80.00   3rd Qu.:32.00
##   Max.   :17.000   Max.   :199.0   Max.   :122.00   Max.   :99.00
##   Insulin      BMI      DiabetesPedigreeFunction      Age
##   Min.   : 0.0   Min.   : 0.00   Min.   :0.0780   Min.   :21.00
##   1st Qu.: 0.0   1st Qu.:27.30   1st Qu.:0.2437   1st Qu.:24.00
##   Median : 30.5   Median :32.00   Median :0.3725   Median :29.00
##   Mean   : 79.8   Mean   :31.99   Mean   :0.4719   Mean   :33.24
##   3rd Qu.:127.2   3rd Qu.:36.60   3rd Qu.:0.6262   3rd Qu.:41.00
##   Max.   :846.0   Max.   :67.10   Max.   :2.4200   Max.   :81.00
##   Outcome
##   Min.   :0.000
##   1st Qu.:0.000
##   Median :0.000
##   Mean   :0.349
##   3rd Qu.:1.000
##   Max.   :1.000
```

Box Plot:

```
par(mfrow = c(2, 2))
par(mar = c(3, 3, 2, 1))

for (i in 1:8) {
  boxplot(diabetes[, i] ~ diabetes$Outcome,
    main = names(diabetes)[i],
    xlab = names(diabetes)[i],
    ylab = "Outcome",
    col = c("green", "yellow"),
    names = c("No Diabetes", "Diabetes"))
}
```





Histogram:

```
par(mfrow=c(3,3))

hist(diabetes$Pregnancies, main = "Histogram of Pregnancies",
     xlab = "Pregnancies", col = "skyblue", border = "darkblue")

hist(diabetes$Glucose, main = "Histogram of Glucose",
     xlab = "Glucose", col = "skyblue", border = "darkblue")

hist(diabetes$BloodPressure, main = "Histogram of BloodPressure",
     xlab = "BloodPressure", col = "skyblue", border = "darkblue")

hist(diabetes$SkinThickness, main = "Histogram of SkinThickness",
     xlab = "SkinThickness", col = "skyblue", border = "darkblue")

hist(diabetes$Insulin, main = "Histogram of Insulin",
     xlab = "Insulin", col = "skyblue", border = "darkblue")

hist(diabetes$BMI, main = "Histogram of BMI",
     xlab = "BMI", col = "skyblue", border = "darkblue")

hist(diabetes$DiabetesPedigreeFunction, main = "Histogram of DiabetesPedigreeFunction",
```

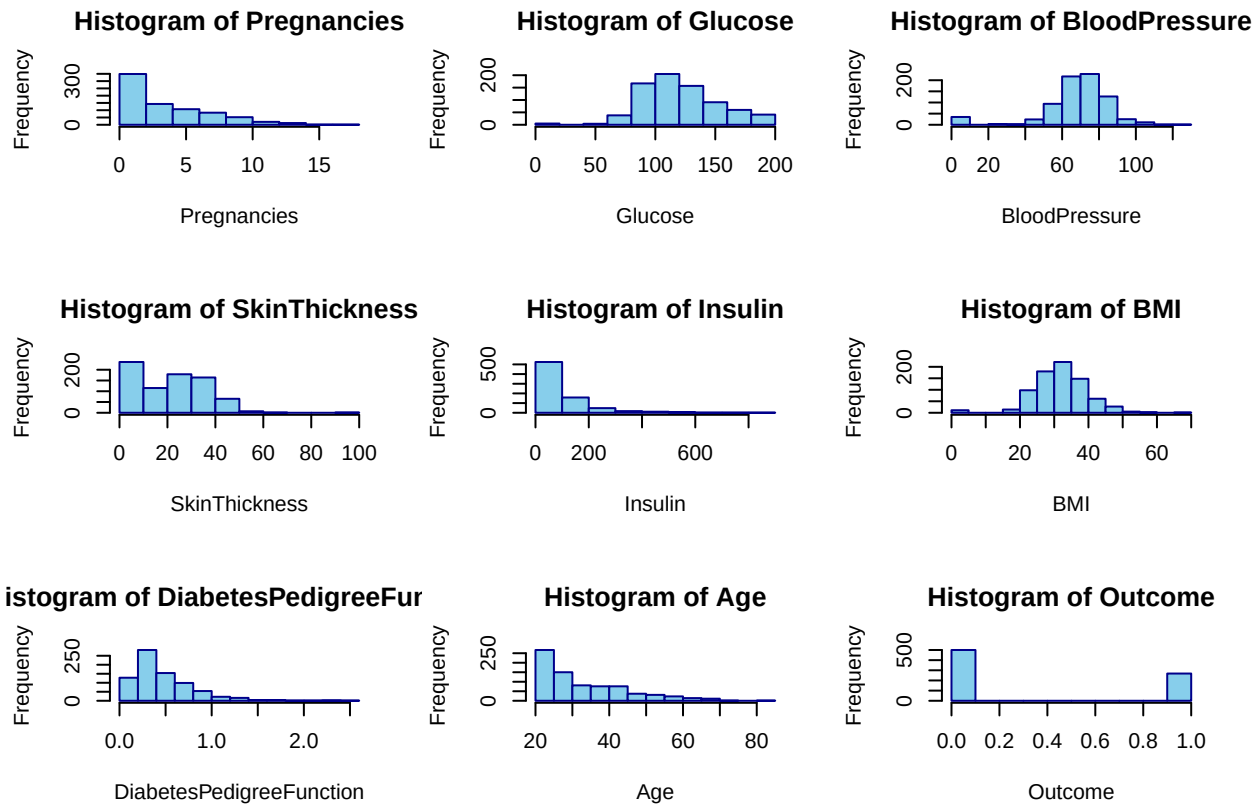
```

xlab = "DiabetesPedigreeFunction", col = "skyblue", border = "darkblue")

hist(diabetes$Age, main = "Histogram of Age",
     xlab = "Age", col = "skyblue", border = "darkblue")

hist(diabetes$Outcome, main = "Histogram of Outcome",
     xlab = "Outcome", col = "skyblue", border = "darkblue")

```



Non-logical Values:

```

# Finding missing values at first:
colSums(is.na(diabetes))

```

```

##           Pregnancies           Glucose           BloodPressure
##                0                0                0
##           SkinThickness           Insulin                BMI
##                0                0                0
## DiabetesPedigreeFunction           Age           Outcome
##                0                0                0

```

```
#Count 0 in each column:
sapply(diabetes, function(x) sum(x == 0, na.rm = TRUE))
```

```
##           Pregnancies           Glucose           BloodPressure
##           111              5              35
##           SkinThickness       Insulin           BMI
##           227             374             11
## DiabetesPedigreeFunction      Age           Outcome
##           0              0              500
```

```
#Count clinically nonsensical 0 only
nonsense_zero_counts <- c(
  Glucose = sum(diabetes$Glucose == 0, na.rm = TRUE),
  BloodPressure = sum(diabetes$BloodPressure == 0, na.rm = TRUE),
  SkinThickness = sum(diabetes$SkinThickness == 0, na.rm = TRUE),
  Insulin = sum(diabetes$Insulin == 0, na.rm = TRUE),
  BMI = sum(diabetes$BMI == 0, na.rm = TRUE)
)

nonsense_zero_counts
```

```
##           Glucose BloodPressure SkinThickness       Insulin       BMI
##           5         35         227         374         11
```

Recode Selected 0 Values to NA:

```
#Recode non-logical 0 values into NA:
diabetes_clean <- diabetes

diabetes_clean$Glucose[diabetes_clean$Glucose == 0] <- NA
diabetes_clean$BloodPressure[diabetes_clean$BloodPressure == 0] <- NA
diabetes_clean$SkinThickness[diabetes_clean$SkinThickness == 0] <- NA
diabetes_clean$Insulin[diabetes_clean$Insulin == 0] <- NA
diabetes_clean$BMI[diabetes_clean$BMI == 0] <- NA

#Check NA counts after recoding:
colSums(is.na(diabetes_clean))
```

```
##           Pregnancies           Glucose           BloodPressure
##           0              5              35
##           SkinThickness       Insulin           BMI
##           227             374             11
## DiabetesPedigreeFunction      Age           Outcome
##           0              0              0
```

% of Missing Value:

```

# % of Missing value after recoding:
missing_count <- colSums(is.na(diabetes_clean))
missing_percentage <- round((missing_count / nrow(diabetes_clean)) * 100, 2)

# Sumummary table of missing value:
missing_summary <- data.frame(
  Variable = names(missing_count),
  Missing_Count = missing_count,
  Missing_Percentage = missing_percentage
)

missing_summary

```

```

##                               Variable Missing_Count
## Pregnancies                   Pregnancies           0
## Glucose                       Glucose             5
## BloodPressure                 BloodPressure        35
## SkinThickness                 SkinThickness       227
## Insulin                       Insulin            374
## BMI                           BMI                11
## DiabetesPedigreeFunction      DiabetesPedigreeFunction  0
## Age                           Age                 0
## Outcome                       Outcome             0
##                               Missing_Percentage
## Pregnancies                   0.00
## Glucose                       0.65
## BloodPressure                 4.56
## SkinThickness                 29.56
## Insulin                       48.70
## BMI                           1.43
## DiabetesPedigreeFunction      0.00
## Age                           0.00
## Outcome                       0.00

```

Median Calculation & Imputation:

```

#Median values for variables with non-logical 0 values:
glucose_median <- median(diabetes_clean$Glucose, na.rm = TRUE)
bloodpressure_median <- median(diabetes_clean$BloodPressure, na.rm = TRUE)
skintickness_median <- median(diabetes_clean$SkinThickness, na.rm = TRUE)
insulin_median <- median(diabetes_clean$Insulin, na.rm = TRUE)
bmi_median <- median(diabetes_clean$BMI, na.rm = TRUE)

#Show median values:
glucose_median

```

```
## [1] 117
```

```
bloodpressure_median
```

```
## [1] 72
```

```
skinthickness_median
```

```
## [1] 29
```

```
insulin_median
```

```
## [1] 125
```

```
bmi_median
```

```
## [1] 32.3
```

```
#Median imputation for NA values:
```

```
diabetes_imputed <- diabetes_clean
```

```
diabetes_imputed$Glucose[is.na(diabetes_imputed$Glucose)] <- glucose_median
```

```
diabetes_imputed$BloodPressure[is.na(diabetes_imputed$BloodPressure)] <- bloodpressure_median
```

```
diabetes_imputed$SkinThickness[is.na(diabetes_imputed$SkinThickness)] <- skinthickness_median
```

```
diabetes_imputed$Insulin[is.na(diabetes_imputed$Insulin)] <- insulin_median
```

```
diabetes_imputed$BMI[is.na(diabetes_imputed$BMI)] <- bmi_median
```

Imputed dataset:

```
# Check the imputed dataset:
```

```
head(diabetes_imputed)
```

```
##   Pregnancies Glucose BloodPressure SkinThickness Insulin  BMI
## 1           6    148           72           35    125 33.6
## 2           1     85           66           29    125 26.6
## 3           8    183           64           29    125 23.3
## 4           1     89           66           23     94 28.1
## 5           0    137           40           35    168 43.1
## 6           5    116           74           29    125 25.6
##   DiabetesPedigreeFunction Age Outcome
## 1                0.627  50         1
## 2                0.351  31         0
## 3                0.672  32         1
## 4                0.167  21         0
## 5                2.288  33         1
## 6                0.201  30         0
```

```
summary(diabetes_imputed)
```

```
##      Pregnancies      Glucose      BloodPressure      SkinThickness
##  Min.   : 0.000    Min.   : 44.00    Min.   : 24.00    Min.   : 7.00
## 1st Qu.: 1.000    1st Qu.: 99.75    1st Qu.: 64.00    1st Qu.:25.00
## Median : 3.000    Median :117.00    Median : 72.00    Median :29.00
## Mean   : 3.845    Mean   :121.66    Mean   : 72.39    Mean   :29.11
## 3rd Qu.: 6.000    3rd Qu.:140.25    3rd Qu.: 80.00    3rd Qu.:32.00
## Max.   :17.000    Max.   :199.00    Max.   :122.00    Max.   :99.00
##      Insulin      BMI      DiabetesPedigreeFunction      Age
##  Min.   : 14.0    Min.   :18.20    Min.   :0.0780    Min.   :21.00
## 1st Qu.:121.5    1st Qu.:27.50    1st Qu.:0.2437    1st Qu.:24.00
## Median :125.0    Median :32.30    Median :0.3725    Median :29.00
## Mean   :140.7    Mean   :32.46    Mean   :0.4719    Mean   :33.24
## 3rd Qu.:127.2    3rd Qu.:36.60    3rd Qu.:0.6262    3rd Qu.:41.00
## Max.   :846.0    Max.   :67.10    Max.   :2.4200    Max.   :81.00
##      Outcome
##  Min.   :0.000
## 1st Qu.:0.000
## Median :0.000
## Mean   :0.349
## 3rd Qu.:1.000
## Max.   :1.000
```

```
# Confirm that no missing values remain:
```

```
colSums(is.na(diabetes_imputed))
```

```
##      Pregnancies      Glucose      BloodPressure
##              0              0              0
##      SkinThickness      Insulin      BMI
##              0              0              0
## DiabetesPedigreeFunction      Age      Outcome
##              0              0              0
```

```
# Export the final cleaned and imputed dataset:
```

```
write.csv(diabetes_imputed,
          "diabetes_imputed.csv",
          row.names = FALSE)
```

```
#visualization of histograms
```

```
library(ggplot2)
```

```
library(tidyr)
```

```
library(dplyr)
```

```
diabetes_long <- diabetes %>%
```

```
  pivot_longer(cols = -Outcome, names_to = "Variable", values_to = "Value")
```

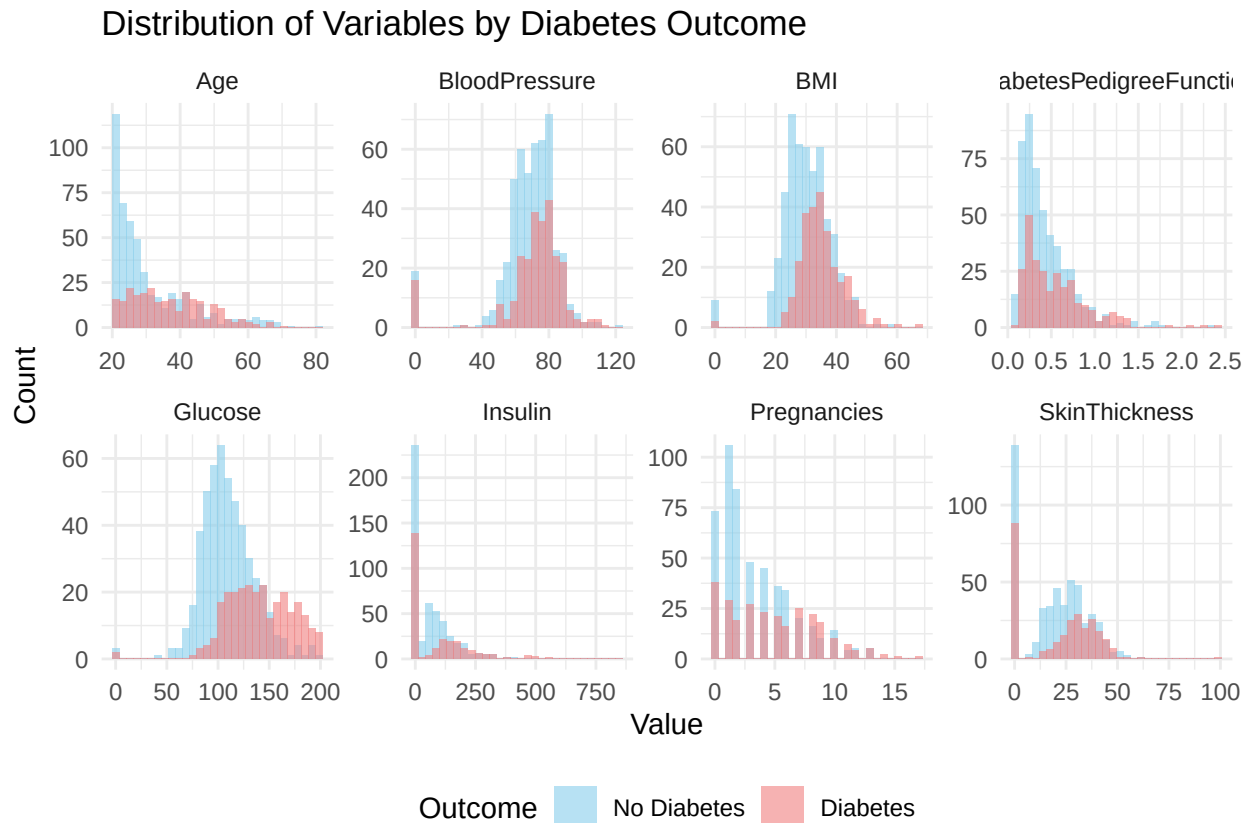
```
ggplot(diabetes_long, aes(x = Value, fill = factor(Outcome))) +
  geom_histogram(alpha = 0.6, position = "identity", bins = 30) +
  facet_wrap(~ Variable, scales = "free", ncol = 4) +
  scale_fill_manual(values = c("skyblue", "lightcoral"),
```



```

      labels = c("No Diabetes", "Diabetes")) +
labs(title = "Distribution of Variables by Diabetes Outcome",
     x = "Value", y = "Count", fill = "Outcome") +
theme_minimal() +
theme(legend.position = "bottom")

```



#visualization of violin plots

```

diabetes_imputed_long <- diabetes_imputed %>%
  pivot_longer(cols = -Outcome, names_to = "Variable", values_to = "Value")

ggplot(diabetes_imputed_long, aes(x = Outcome, y = Value, fill = Outcome)) +
  geom_violin(trim = FALSE, alpha = 0.6) +
  geom_boxplot(width = 0.15, fill = "white", alpha = 0.8) +
  facet_wrap(~ Variable, scales = "free_y") +
  labs(x = "Outcome", y = "Value",
       title = "Violin Plot of Variables by Diabetes Outcome") +
  scale_fill_manual(values = c("green", "red")) +
  theme_minimal()

```

```

## Warning: The following aesthetics were dropped during statistical transformation: fill.
## i This can happen when ggplot fails to infer the correct grouping structure in
##   the data.
## i Did you forget to specify a 'group' aesthetic or to convert a numerical
##   variable into a factor?

```

```

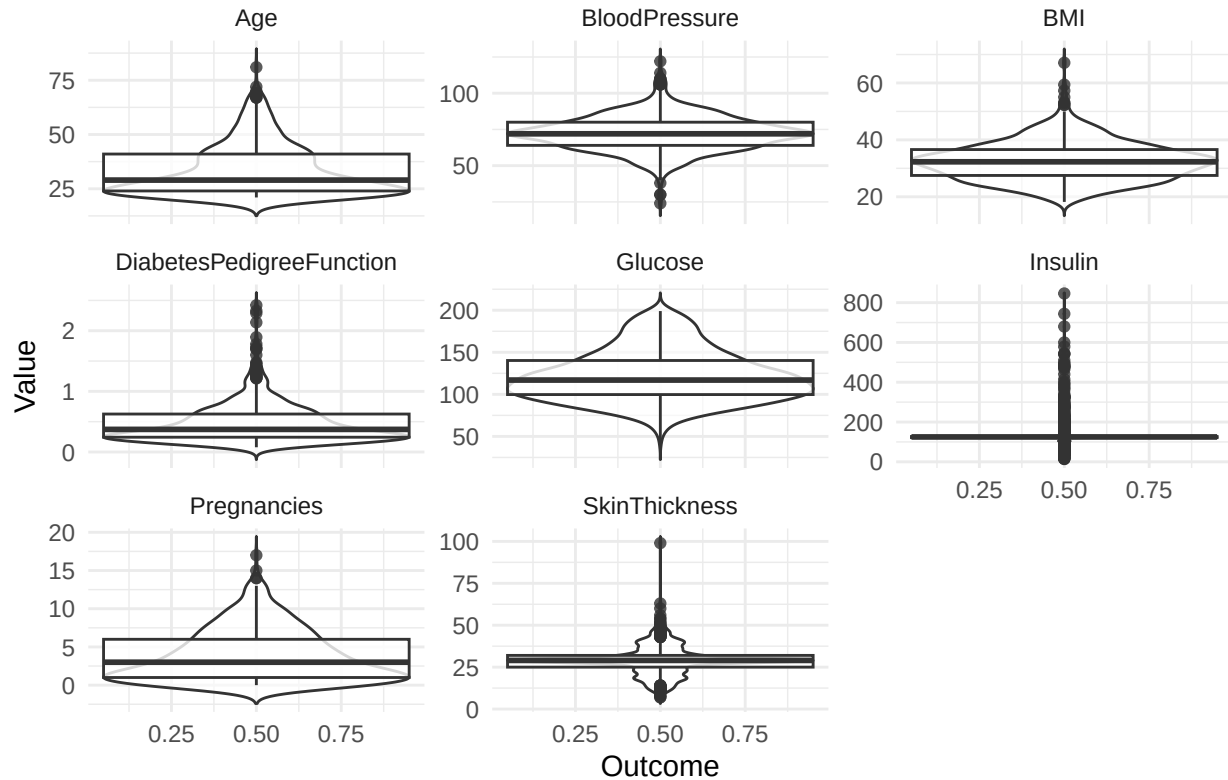
## The following aesthetics were dropped during statistical transformation: fill.
## i This can happen when ggplot fails to infer the correct grouping structure in
## the data.
## i Did you forget to specify a 'group' aesthetic or to convert a numerical
## variable into a factor?
## The following aesthetics were dropped during statistical transformation: fill.
## i This can happen when ggplot fails to infer the correct grouping structure in
## the data.
## i Did you forget to specify a 'group' aesthetic or to convert a numerical
## variable into a factor?
## The following aesthetics were dropped during statistical transformation: fill.
## i This can happen when ggplot fails to infer the correct grouping structure in
## the data.
## i Did you forget to specify a 'group' aesthetic or to convert a numerical
## variable into a factor?
## The following aesthetics were dropped during statistical transformation: fill.
## i This can happen when ggplot fails to infer the correct grouping structure in
## the data.
## i Did you forget to specify a 'group' aesthetic or to convert a numerical
## variable into a factor?
## The following aesthetics were dropped during statistical transformation: fill.
## i This can happen when ggplot fails to infer the correct grouping structure in
## the data.
## i Did you forget to specify a 'group' aesthetic or to convert a numerical
## variable into a factor?
## The following aesthetics were dropped during statistical transformation: fill.
## i This can happen when ggplot fails to infer the correct grouping structure in
## the data.
## i Did you forget to specify a 'group' aesthetic or to convert a numerical
## variable into a factor?

## Warning: Orientation is not uniquely specified when both the x and y aesthetics are
## continuous. Picking default orientation 'x'.

## Warning: Continuous x aesthetic
## i did you forget 'aes(group = ...)'?

```

Violin Plot of Variables by Diabetes Outcome



Correlation Matrix:

```
library(corrplot)
```

```
## corrplot 0.95 loaded
```

```
#Select only predictor variables only
core_vars <- c("Pregnancies", "Glucose", "BloodPressure", "SkinThickness",
               "Insulin", "BMI", "DiabetesPedigreeFunction", "Age")
```

```
cat("x:", core_vars, "\n")
```

```
## x: Pregnancies Glucose BloodPressure SkinThickness Insulin BMI DiabetesPedigreeFunction Age
```

```
cat("y:", "Outcome", "\n")
```

```
## y: Outcome
```

```
#Correlation matrix using imputed data
cor_mat <- cor(diabetes_imputed[, core_vars], use = "pairwise.complete.obs")
```

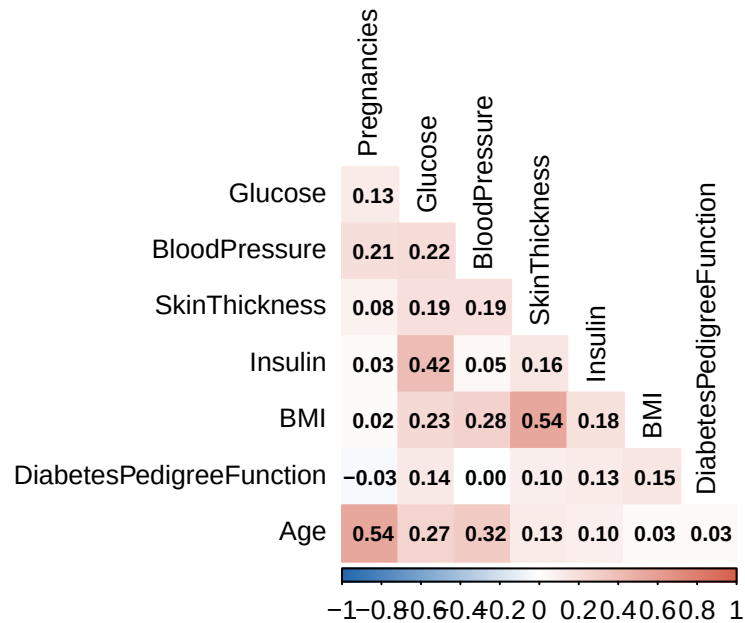
```
# Correlation matrix values:
round(cor_mat, 2)
```

```
##              Pregnancies Glucose BloodPressure SkinThickness
## Pregnancies          1.00   0.13           0.21           0.08
## Glucose              0.13   1.00           0.22           0.19
## BloodPressure        0.21   0.22           1.00           0.19
## SkinThickness        0.08   0.19           0.19           1.00
## Insulin              0.03   0.42           0.05           0.16
## BMI                  0.02   0.23           0.28           0.54
## DiabetesPedigreeFunction -0.03  0.14           0.00           0.10
## Age                  0.54   0.27           0.32           0.13
##
##              Insulin  BMI DiabetesPedigreeFunction  Age
## Pregnancies      0.03 0.02                    -0.03 0.54
## Glucose          0.42 0.23                    0.14 0.27
## BloodPressure    0.05 0.28                    0.00 0.32
## SkinThickness    0.16 0.54                    0.10 0.13
## Insulin          1.00 0.18                    0.13 0.10
## BMI              0.18 1.00                    0.15 0.03
## DiabetesPedigreeFunction 0.13 0.15                1.00 0.03
## Age              0.10 0.03                    0.03 1.00
```

```
# Visualize correlation plot:
```

```
corrplot(cor_mat,
  method = "color",
  type = "lower",
  diag = FALSE,
  addCoef.col = "black",
  tl.col = "black",
  tl.cex = 0.8,
  number.cex = 0.7,
  col = colorRampPalette(c("#2166AC", "white", "#D6604D"))(200),
  main = "Correlation Plot of Diabetes Variables",
  mar = c(0, 0, 2, 0))
```

Correlation Plot of Diabetes Variables



Multicollinearity:

1. Core Model (exclude Isulin & Skin Thickness):

```
library(car)
```

```
## Loading required package: carData
```

```
##
```

```
## Attaching package: 'car'
```

```
## The following object is masked from 'package:dplyr':
```

```
##
```

```
## recode
```

```
## The following object is masked from 'package:purrr':
```

```
##
```

```
## some
```

```

# Fit logistic regression model using the imputed dataset (without Isulin & Skin Thickness due to overl
model_core <- glm(Outcome ~ Pregnancies + Glucose + BloodPressure + BMI +
                  DiabetesPedigreeFunction + Age,
                  data = diabetes_imputed,
                  family = binomial)

# Show model summary
summary(model_core)

```

```

##
## Call:
## glm(formula = Outcome ~ Pregnancies + Glucose + BloodPressure +
##      BMI + DiabetesPedigreeFunction + Age, family = binomial,
##      data = diabetes_imputed)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)   -9.052181    0.808508  -11.196 < 2e-16 ***
## Pregnancies     0.125038    0.032300   3.871 0.000108 ***
## Glucose         0.036388    0.003586  10.149 < 2e-16 ***
## BloodPressure  -0.008596    0.008551  -1.005 0.314784
## BMI             0.094272    0.015373   6.132 8.67e-10 ***
## DiabetesPedigreeFunction 0.861329    0.296177   2.908 0.003636 **
## Age            0.013242    0.009485   1.396 0.162685
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##    Null deviance: 993.48  on 767  degrees of freedom
## Residual deviance: 713.96  on 761  degrees of freedom
## AIC: 727.96
##
## Number of Fisher Scoring iterations: 5

```

```

# Calculate Variance Inflation Factor (VIF)
vif(model_core)

```

```

##              Pregnancies              Glucose              BloodPressure
##              1.414826              1.055367              1.213066
##              BMI DiabetesPedigreeFunction              Age
##              1.119365              1.011073              1.528187

```

2. Full Model:

```

# Full model including SkinThickness and Insulin:
model_full <- glm(Outcome ~ Pregnancies + Glucose + BloodPressure +
                  SkinThickness + Insulin + BMI +
                  DiabetesPedigreeFunction + Age,
                  data = diabetes_imputed,

```

```

family = binomial)

# Show model summary
summary(model_full)

##
## Call:
## glm(formula = Outcome ~ Pregnancies + Glucose + BloodPressure +
##      SkinThickness + Insulin + BMI + DiabetesPedigreeFunction +
##      Age, family = binomial, data = diabetes_imputed)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)    -9.108966   0.813525 -11.197 < 2e-16 ***
## Pregnancies      0.124778   0.032420   3.849 0.000119 ***
## Glucose          0.037855   0.003902   9.703 < 2e-16 ***
## BloodPressure   -0.009373   0.008578  -1.093 0.274540
## SkinThickness    0.003451   0.013154   0.262 0.793074
## Insulin         -0.001172   0.001132  -1.035 0.300627
## BMI              0.094252   0.017893   5.268 1.38e-07 ***
## DiabetesPedigreeFunction 0.875858   0.296740   2.952 0.003161 **
## Age              0.013028   0.009506   1.371 0.170518
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##    Null deviance: 993.48  on 767  degrees of freedom
## Residual deviance: 712.84  on 759  degrees of freedom
## AIC: 730.84
##
## Number of Fisher Scoring iterations: 5

# Calculate VIF for the full model
vif(model_full)

```

```

##              Pregnancies              Glucose              BloodPressure
##              1.417904              1.246326              1.221851
##              SkinThickness              Insulin              BMI
##              1.364557              1.210216              1.514127
## DiabetesPedigreeFunction              Age
##              1.014209              1.532258

```

The VIF results showed that multicollinearity was low in both models, so overlap between variables was not a major issue.

So, we did not leave out **SkinThickness** and **Insulin** because of multicollinearity. They were excluded from the main model because they had many unrealistic 0 values, needed a lot of imputation, and did not improve the model. Therefore, the core model (without **SkinThickness** and **Insulin**) was used as the main training model, while the full model was only kept for comparison.

Train-test data splitting:

```
# Partition the dataset
library(caret)

## Loading required package: lattice

##
## Attaching package: 'caret'

## The following object is masked from 'package:purrr':
##
## lift

set.seed(7835)

# Convert outcome to factor for classification tree
diabetes_imputed$Outcome <- factor(diabetes_imputed$Outcome,
                                   levels = c(0, 1),
                                   labels = c("NoDiabetes", "Diabetes"))

# Stratified split to preserve class balance
train_index <- createDataPartition(diabetes_imputed$Outcome,
                                    p = 0.8,
                                    list = FALSE)

train <- diabetes_imputed[train_index, ]
test  <- diabetes_imputed[-train_index, ]

# Check split sizes
nrow(train)

## [1] 615

nrow(test)

## [1] 153

# Check class distribution
prop.table(table(train$Outcome))

##
## NoDiabetes Diabetes
## 0.6504065 0.3495935

prop.table(table(test$Outcome))

##
## NoDiabetes Diabetes
## 0.6535948 0.3464052
```



```
#Setup for selection
```

```
#original model with 8 variables
full_model <- glm(Outcome ~ ., data = train, family = binomial)

#empty model for later selections
null_model <- glm(Outcome ~ 1, data = train, family = binomial)

summary(full_model)
```

```
##
## Call:
## glm(formula = Outcome ~ ., family = binomial, data = train)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)   -9.0166417  0.8980170 -10.041  < 2e-16 ***
## Pregnancies     0.1137998  0.0355618   3.200  0.00137 **
## Glucose         0.0396505  0.0043929   9.026  < 2e-16 ***
## BloodPressure  -0.0141881  0.0094451  -1.502  0.13305
## SkinThickness   0.0168289  0.0151231   1.113  0.26580
## Insulin        -0.0009835  0.0012632  -0.779  0.43621
## BMI             0.0806186  0.0195058   4.133 3.58e-05 ***
## DiabetesPedigreeFunction 0.6973259  0.3432529   2.032  0.04220 *
## Age            0.0182304  0.0106394   1.713  0.08662 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##    Null deviance: 796.05  on 614  degrees of freedom
## Residual deviance: 561.92  on 606  degrees of freedom
## AIC: 579.92
##
## Number of Fisher Scoring iterations: 5
```

```
summary(null_model)
```

```
##
## Call:
## glm(formula = Outcome ~ 1, family = binomial, data = train)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)  -0.62083    0.08456  -7.341 2.11e-13 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##    Null deviance: 796.05  on 614  degrees of freedom
## Residual deviance: 796.05  on 614  degrees of freedom
```

```
## AIC: 798.05
##
## Number of Fisher Scoring iterations: 4
```

```
#forward selection
```

```
forward_model <- step(
  null_model,
  scope = list(lower = null_model, upper = full_model),
  direction = "forward",
  trace = 0 #show with 1, hide with 0
)

summary(forward_model)
```

```
##
## Call:
## glm(formula = Outcome ~ Glucose + BMI + Pregnancies + DiabetesPedigreeFunction +
##      Age + BloodPressure, family = binomial, data = train)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)    -8.850487   0.883916 -10.013 < 2e-16 ***
## Glucose         0.038339   0.004047   9.473 < 2e-16 ***
## BMI            0.089607   0.016736   5.354 8.6e-08 ***
## Pregnancies    0.116407   0.035475   3.281 0.00103 **
## DiabetesPedigreeFunction 0.701665   0.342548   2.048 0.04052 *
## Age            0.018568   0.010641   1.745 0.08098 .
## BloodPressure  -0.013639   0.009432  -1.446 0.14816
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 796.05  on 614  degrees of freedom
## Residual deviance: 563.72  on 608  degrees of freedom
## AIC: 577.72
##
## Number of Fisher Scoring iterations: 5
```

```
#backward elimination
```

```
backward_model <- step(
  full_model,
  direction = "backward",
  trace = 0
)

summary(backward_model)
```

```
##
```

```
## Call:
## glm(formula = Outcome ~ Pregnancies + Glucose + BloodPressure +
##      BMI + DiabetesPedigreeFunction + Age, family = binomial,
##      data = train)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)    -8.850487   0.883916 -10.013 < 2e-16 ***
## Pregnancies     0.116407   0.035475   3.281 0.00103 **
## Glucose         0.038339   0.004047   9.473 < 2e-16 ***
## BloodPressure  -0.013639   0.009432  -1.446 0.14816
## BMI             0.089607   0.016736   5.354 8.6e-08 ***
## DiabetesPedigreeFunction 0.701665   0.342548   2.048 0.04052 *
## Age            0.018568   0.010641   1.745 0.08098 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 796.05  on 614  degrees of freedom
## Residual deviance: 563.72  on 608  degrees of freedom
## AIC: 577.72
##
## Number of Fisher Scoring iterations: 5
```

#Stepwise selection

```
stepwise_model <- step(
  null_model,
  scope = list(lower = null_model, upper = full_model),
  direction = "both",
  trace = 0
)

summary(stepwise_model)
```

```
##
## Call:
## glm(formula = Outcome ~ Glucose + BMI + Pregnancies + DiabetesPedigreeFunction +
##      Age + BloodPressure, family = binomial, data = train)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)    -8.850487   0.883916 -10.013 < 2e-16 ***
## Glucose         0.038339   0.004047   9.473 < 2e-16 ***
## BMI             0.089607   0.016736   5.354 8.6e-08 ***
## Pregnancies     0.116407   0.035475   3.281 0.00103 **
## DiabetesPedigreeFunction 0.701665   0.342548   2.048 0.04052 *
## Age            0.018568   0.010641   1.745 0.08098 .
## BloodPressure  -0.013639   0.009432  -1.446 0.14816
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
##
## (Dispersion parameter for binomial family taken to be 1)
##
## Null deviance: 796.05 on 614 degrees of freedom
## Residual deviance: 563.72 on 608 degrees of freedom
## AIC: 577.72
##
## Number of Fisher Scoring iterations: 5
```

Odds Ratio (OR) calculations with table

```
summary(stepwise_model)
```

```
##
## Call:
## glm(formula = Outcome ~ Glucose + BMI + Pregnancies + DiabetesPedigreeFunction +
## Age + BloodPressure, family = binomial, data = train)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)   -8.850487   0.883916 -10.013 < 2e-16 ***
## Glucose         0.038339   0.004047   9.473 < 2e-16 ***
## BMI            0.089607   0.016736   5.354 8.6e-08 ***
## Pregnancies    0.116407   0.035475   3.281 0.00103 **
## DiabetesPedigreeFunction 0.701665   0.342548   2.048 0.04052 *
## Age            0.018568   0.010641   1.745 0.08098 .
## BloodPressure  -0.013639   0.009432  -1.446 0.14816
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
## Null deviance: 796.05 on 614 degrees of freedom
## Residual deviance: 563.72 on 608 degrees of freedom
## AIC: 577.72
##
## Number of Fisher Scoring iterations: 5
```

```
exp_coef <- exp(coef(stepwise_model))
exp_ci <- exp(confint(stepwise_model))
```

```
## Waiting for profiling to be done...
```

```
coef_table <- data.frame(
  Variable = names(coef(stepwise_model)),
  Coefficient = round(coef(stepwise_model), 4),
  Odds_Ratio = round(exp_coef, 3),
  CI_lower = round(exp_ci[, 1], 3),
  CI_upper = round(exp_ci[, 2], 3)
)

print(coef_table)
```

	Variable	Coefficient	Odds_Ratio
## (Intercept)	(Intercept)	-8.8505	0.000
## Glucose	Glucose	0.0383	1.039
## BMI	BMI	0.0896	1.094
## Pregnancies	Pregnancies	0.1164	1.123
## DiabetesPedigreeFunction	DiabetesPedigreeFunction	0.7017	2.017
## Age	Age	0.0186	1.019
## BloodPressure	BloodPressure	-0.0136	0.986

	CI_lower	CI_upper
## (Intercept)	0.000	0.001
## Glucose	1.031	1.048
## BMI	1.059	1.131
## Pregnancies	1.049	1.205
## DiabetesPedigreeFunction	1.037	3.977
## Age	0.998	1.040
## BloodPressure	0.968	1.005

#Once glucose increase 1mg/dL, the risk of diabetes increase 3.9% (explanation above) All predictors except age and blood pressure had 95% confidence intervals that did not contain 1, confirming their statistical significance.

#Choose the stepwise selection and test

```

pred_prob <- predict(stepwise_model,
                     newdata = test,
                     type = "response")

pred_class <- ifelse(pred_prob > 0.5, "Diabetes", "NoDiabetes")
pred_class <- factor(pred_class, levels = c("NoDiabetes", "Diabetes"))

accuracy <- mean(pred_class == test$Outcome)
cat("Accuracy:", round(accuracy, 4), "\n")

```

Accuracy: 0.7255

```

conf_matrix <- table(Predicted = pred_class, Actual = test$Outcome)
print(conf_matrix)

```

	Actual	
Predicted	NoDiabetes	Diabetes
NoDiabetes	86	28
Diabetes	14	25

```

sensitivity <- conf_matrix["Diabetes", "Diabetes"] / sum(conf_matrix[, "Diabetes"])
cat("Sensitivity:", round(sensitivity, 4), "\n")

```

Sensitivity: 0.4717

```

specificity <- conf_matrix["NoDiabetes", "NoDiabetes"] / sum(conf_matrix[, "NoDiabetes"])
cat("Specificity:", round(specificity, 4), "\n")

```

Specificity: 0.86

AUC model for LR

```
library(pROC)

## Type 'citation("pROC")' for a citation.

##
## Attaching package: 'pROC'

## The following objects are masked from 'package:stats':
##
##     cov, smooth, var

test_outcome_num <- ifelse(test$Outcome == "Diabetes", 1, 0)

roc_obj <- roc(test_outcome_num, pred_prob)

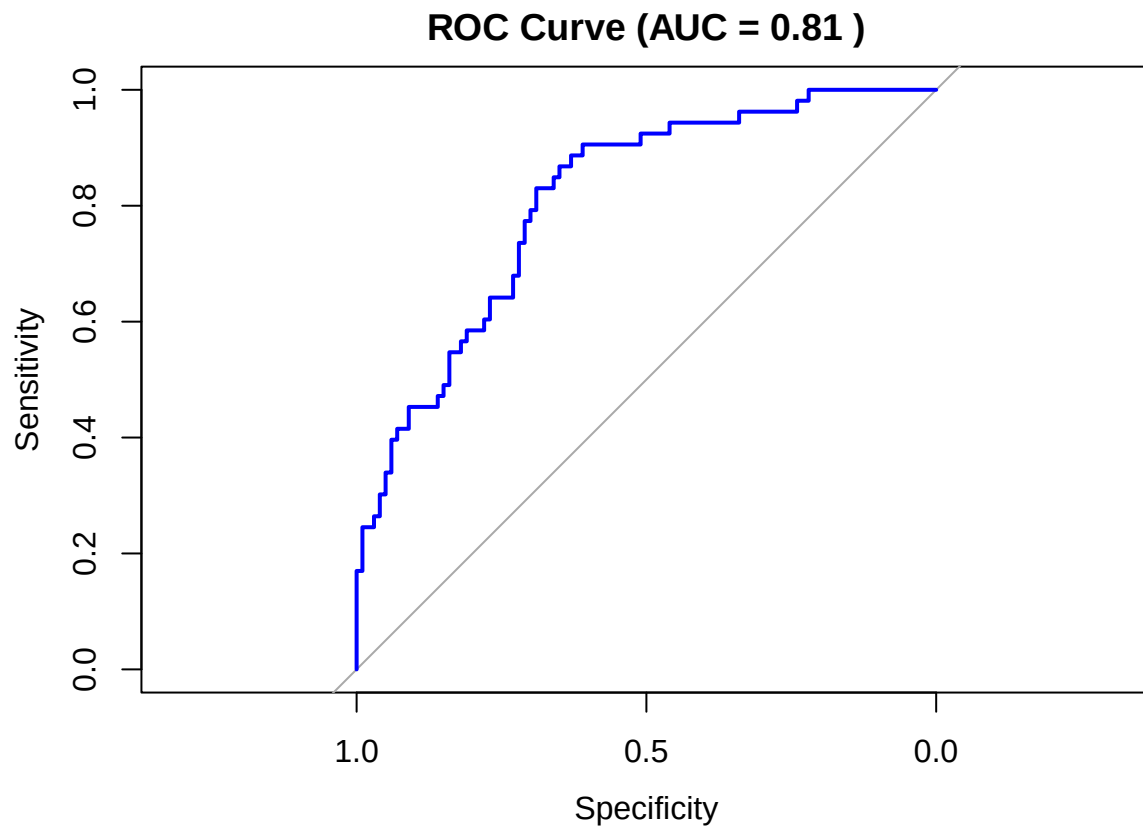
## Setting levels: control = 0, case = 1

## Setting direction: controls < cases

auc_value <- auc(roc_obj)
cat("AUC:", round(auc_value, 4), "\n")

## AUC: 0.8102

plot(roc_obj, col = "blue", lwd = 2,
      main = paste("ROC Curve (AUC =", round(auc_value, 3), ")"))
```



Fit a classification tree:

```
# Classification tree:
library(rpart)
library(rpart.plot)

# Specify target and input variables
formula <- Outcome ~ .

# Fit classification tree on training set
dtree_diabetes <- rpart(formula,
                        data = train,
                        method = "class")

# Print the tree
print(dtree_diabetes)
```

```
## n= 615
##
## node), split, n, loss, yval, (yprob)
##      * denotes terminal node
##
##  1) root 615 215 NoDiabetes (0.65040650 0.34959350)
```

```
##      2) Glucose< 143.5 475 110 NoDiabetes (0.76842105 0.23157895)
##      4) BMI< 27.35 132 5 NoDiabetes (0.96212121 0.03787879) *
##      5) BMI>=27.35 343 105 NoDiabetes (0.69387755 0.30612245)
##      10) Glucose< 106.5 155 24 NoDiabetes (0.84516129 0.15483871)
##      20) DiabetesPedigreeFunction< 0.6575 129 14 NoDiabetes (0.89147287 0.10852713) *
##      21) DiabetesPedigreeFunction>=0.6575 26 10 NoDiabetes (0.61538462 0.38461538)
##      42) Insulin< 122 17 3 NoDiabetes (0.82352941 0.17647059) *
##      43) Insulin>=122 9 2 Diabetes (0.22222222 0.77777778) *
##      11) Glucose>=106.5 188 81 NoDiabetes (0.56914894 0.43085106)
##      22) Age< 30.5 103 29 NoDiabetes (0.71844660 0.28155340) *
##      23) Age>=30.5 85 33 Diabetes (0.38823529 0.61176471)
##      46) DiabetesPedigreeFunction< 0.52 49 24 NoDiabetes (0.51020408 0.48979592)
##      92) BloodPressure>=75 27 9 NoDiabetes (0.66666667 0.33333333)
##      184) Insulin< 127.5 19 3 NoDiabetes (0.84210526 0.15789474) *
##      185) Insulin>=127.5 8 2 Diabetes (0.25000000 0.75000000) *
##      93) BloodPressure< 75 22 7 Diabetes (0.31818182 0.68181818) *
##      47) DiabetesPedigreeFunction>=0.52 36 8 Diabetes (0.22222222 0.77777778) *
##      3) Glucose>=143.5 140 35 Diabetes (0.25000000 0.75000000) *
```

```
# Show complexity parameter table
printcp(dtree_diabetes)
```

```
##
## Classification tree:
## rpart(formula = formula, data = train, method = "class")
##
## Variables actually used in tree construction:
## [1] Age BloodPressure BMI
## [4] DiabetesPedigreeFunction Glucose Insulin
##
## Root node error: 215/615 = 0.34959
##
## n= 615
##
##      CP nsplit rel error  xerror  xstd
## 1 0.325581 0 1.00000 1.00000 0.055001
## 2 0.029457 1 0.67442 0.73023 0.050293
## 3 0.020930 4 0.58605 0.69767 0.049533
## 4 0.018605 6 0.54419 0.69302 0.049421
## 5 0.011628 7 0.52558 0.70698 0.049755
## 6 0.010000 9 0.50233 0.71628 0.049973
```

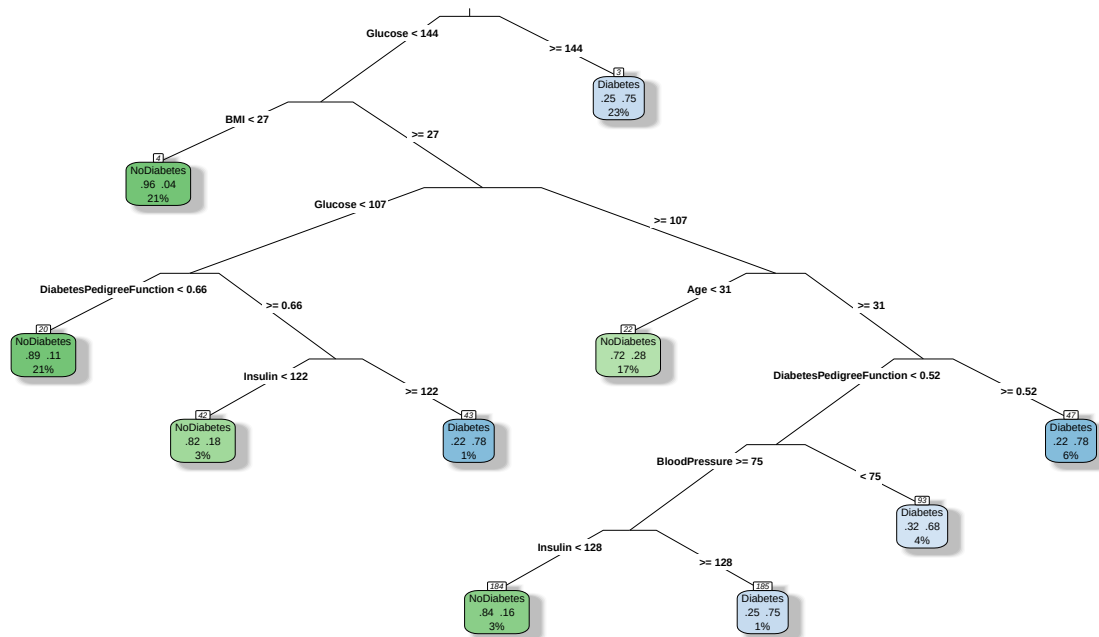
Visualize the decision tree

```
# Plot the classification tree
rpart.plot(dtree_diabetes,
  type = 3,
  fallen.leaves = FALSE,
  extra = 104,
  box.palette = "GnBu",
  branch.lty = 1,
```



```
shadow.col = "gray",
nn = TRUE,
main = "Classification Tree for Diabetes Prediction")
```

Classification Tree for Diabetes Prediction



Check variable importance

```
# Variable importance
dtree_diabetes$variable.importance
```

```
##           Glucose           Insulin           BMI
##           73.930912          18.839213          17.708705
##           Age DiabetesPedigreeFunction Pregnancies
##           13.406620           10.286122           9.703609
##           BloodPressure       SkinThickness
##           5.413807           5.275379
```

Predictions on train and test sets

```

# Predict on training and test sets
train$predicted_tree <- predict(dtree_diabetes,
                               train,
                               type = "class")

test$predicted_tree <- predict(dtree_diabetes,
                              test,
                              type = "class")

# Confusion matrix on test set
tree_cm <- confusionMatrix(test$predicted_tree,
                           test$Outcome,
                           positive = "Diabetes")

tree_cm

```

```

## Confusion Matrix and Statistics
##
##              Reference
## Prediction  NoDiabetes Diabetes
## NoDiabetes      78      19
## Diabetes       22      34
##
##              Accuracy : 0.732
##              95% CI : (0.6545, 0.8003)
## No Information Rate : 0.6536
## P-Value [Acc > NIR] : 0.02367
##
##              Kappa : 0.416
##
## Mcnemar's Test P-Value : 0.75478
##
##              Sensitivity : 0.6415
##              Specificity : 0.7800
##              Pos Pred Value : 0.6071
##              Neg Pred Value : 0.8041
##              Prevalence : 0.3464
##              Detection Rate : 0.2222
##              Detection Prevalence : 0.3660
##              Balanced Accuracy : 0.7108
##
##              'Positive' Class : Diabetes
##

```

```

# Main metrics for original tree
tree_accuracy <- tree_cm$overall["Accuracy"]
tree_sensitivity <- tree_cm$byClass["Sensitivity"]
tree_specificity <- tree_cm$byClass["Specificity"]

tree_accuracy

```

```

## Accuracy
## 0.7320261

```

```
tree_sensitivity
```

```
## Sensitivity  
## 0.6415094
```

```
tree_specificity
```

```
## Specificity  
## 0.78
```

```
# False positives and false negatives  
tree_table <- table(Predicted = test$predicted_tree,  
                    Actual = test$Outcome)
```

```
tree_table
```

```
##           Actual  
## Predicted NoDiabetes Diabetes  
## NoDiabetes      78      19  
## Diabetes       22      34
```

```
tree_false_positives <- tree_table["Diabetes", "NoDiabetes"]  
tree_false_negatives <- tree_table["NoDiabetes", "Diabetes"]
```

```
tree_false_positives
```

```
## [1] 22
```

```
tree_false_negatives
```

```
## [1] 19
```

Prune the tree:

```
# Find best CP value using lowest xerror  
best_cp <- dtree_diabetes$cptable[which.min(dtree_diabetes$cptable[, "xerror"]), "CP"]  
best_cp
```

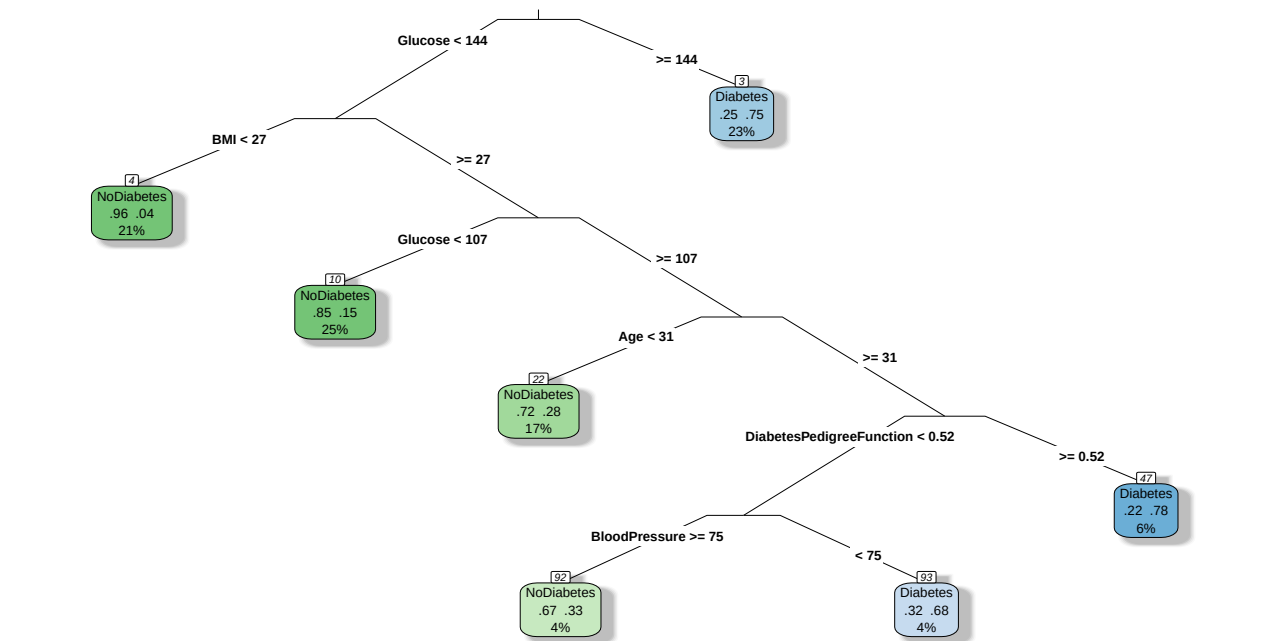
```
## [1] 0.01860465
```

```
# Prune the tree  
pruned_dtree_diabetes <- prune(dtree_diabetes, cp = best_cp)
```

```
# Plot pruned tree  
rpart.plot(pruned_dtree_diabetes,  
            type = 3,  
            fallen.leaves = FALSE,
```

```
extra = 104,  
box.palette = "GnBu",  
branch.lty = 1,  
shadow.col = "gray",  
nn = TRUE,  
main = "Pruned Classification Tree for Diabetes Prediction")
```

Pruned Classification Tree for Diabetes Prediction



Evaluate pruned tree:

[illegible]

```
## Confusion Matrix and Statistics
```

```
##
##           Reference
## Prediction  NoDiabetes Diabetes
##   NoDiabetes      81      24
##   Diabetes       19      29
##
##           Accuracy : 0.719
##           95% CI : (0.6407, 0.7886)
##   No Information Rate : 0.6536
##   P-Value [Acc > NIR] : 0.05152
##
##           Kappa : 0.3653
##
## Mcnemar's Test P-Value : 0.54187
##
##           Sensitivity : 0.5472
##           Specificity : 0.8100
##   Pos Pred Value : 0.6042
##   Neg Pred Value : 0.7714
##   Prevalence : 0.3464
##   Detection Rate : 0.1895
##   Detection Prevalence : 0.3137
##   Balanced Accuracy : 0.6786
##
##   'Positive' Class : Diabetes
##
```

```
# Main metrics for pruned tree
pruned_accuracy <- pruned_cm$overall["Accuracy"]
pruned_sensitivity <- pruned_cm$byClass["Sensitivity"]
pruned_specificity <- pruned_cm$byClass["Specificity"]

pruned_accuracy
```

```
## Accuracy
## 0.7189542
```

```
pruned_sensitivity
```

```
## Sensitivity
## 0.5471698
```

```
pruned_specificity
```

```
## Specificity
## 0.81
```

Compare original tree vs pruned tree:

```
# Summary comparison
tree_comparison <- data.frame(
  Model = c("Original Decision Tree", "Pruned Decision Tree"),
  Accuracy = c(tree_accuracy, pruned_accuracy),
  Sensitivity = c(tree_sensitivity, pruned_sensitivity),
  Specificity = c(tree_specificity, pruned_specificity)
)

tree_comparison
```

```
##              Model Accuracy Sensitivity Specificity
## 1 Original Decision Tree 0.7320261    0.6415094      0.78
## 2   Pruned Decision Tree 0.7189542    0.5471698      0.81
```

```
library(pROC)

tree_prob <- predict(dtrees_diabetes, newdata = test, type = "prob")
test_diabetes_prob <- tree_prob[, "Diabetes"]
test_outcome_num <- ifelse(test$Outcome == "Diabetes", 1, 0)

tree_roc <- roc(test_outcome_num, test_diabetes_prob)
```

```
## Setting levels: control = 0, case = 1
```

```
## Setting direction: controls < cases
```

```
tree_auc <- auc(tree_roc)
cat("Original Decision Tree AUC:", round(tree_auc, 4), "\n")
```

```
## Original Decision Tree AUC: 0.7479
```

```
pruned_prob <- predict(pruned_dtrees_diabetes, newdata = test, type = "prob")
pruned_diabetes_prob <- pruned_prob[, "Diabetes"]
```

```
pruned_roc <- roc(test_outcome_num, pruned_diabetes_prob)
```

```
## Setting levels: control = 0, case = 1
```

```
## Setting direction: controls < cases
```

```
pruned_auc <- auc(pruned_roc)
cat("Pruned Decision Tree AUC:", round(pruned_auc, 4), "\n")
```

```
## Pruned Decision Tree AUC: 0.7538
```

```

plot(tree_roc, col = "blue", lwd = 2,
     main = "ROC Curves: Decision Trees vs Logistic Regression")
lines(pruned_roc, col = "red", lwd = 2)
lines(roc_obj, col = "darkgreen", lwd = 2)

legend("bottomright",
     legend = c(paste("Original Tree (AUC =", round(tree_auc, 3), ")"),
               paste("Pruned Tree (AUC =", round(pruned_auc, 3), ")"),
               paste("Logistic Regression (AUC =", round(auc_value, 3), ")")),
     col = c("blue", "red", "darkgreen"),
     lwd = 2)

```

